

Computer interfacing (CI)

2. PIC18 architecture

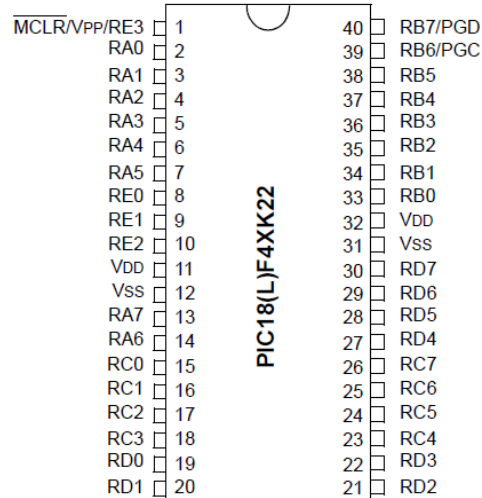
2.1 The 18F45K22 device

From the **Datasheet** (see CI's documentation):

Chip pinout:

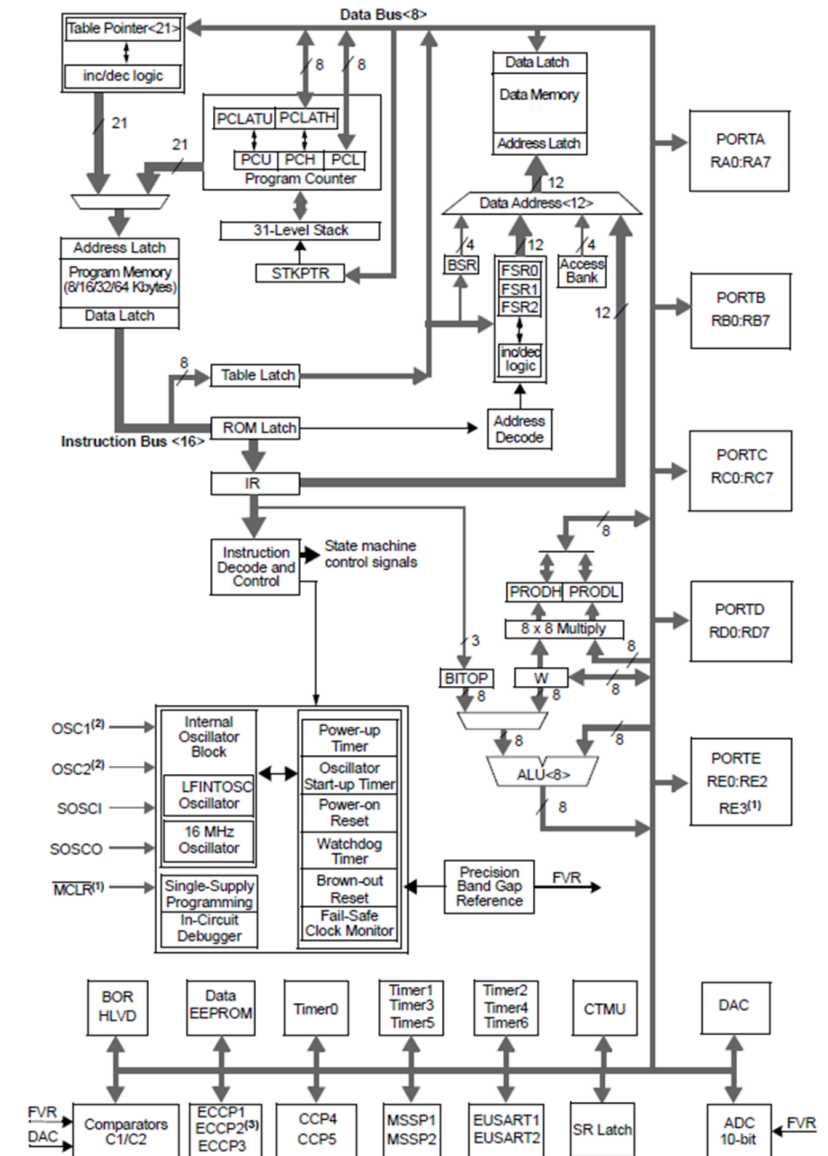
which pin is connected to each function?

40-PIN PDIP DIAGRAM



Internal block diagram:

Showing main device functions and interconnections.



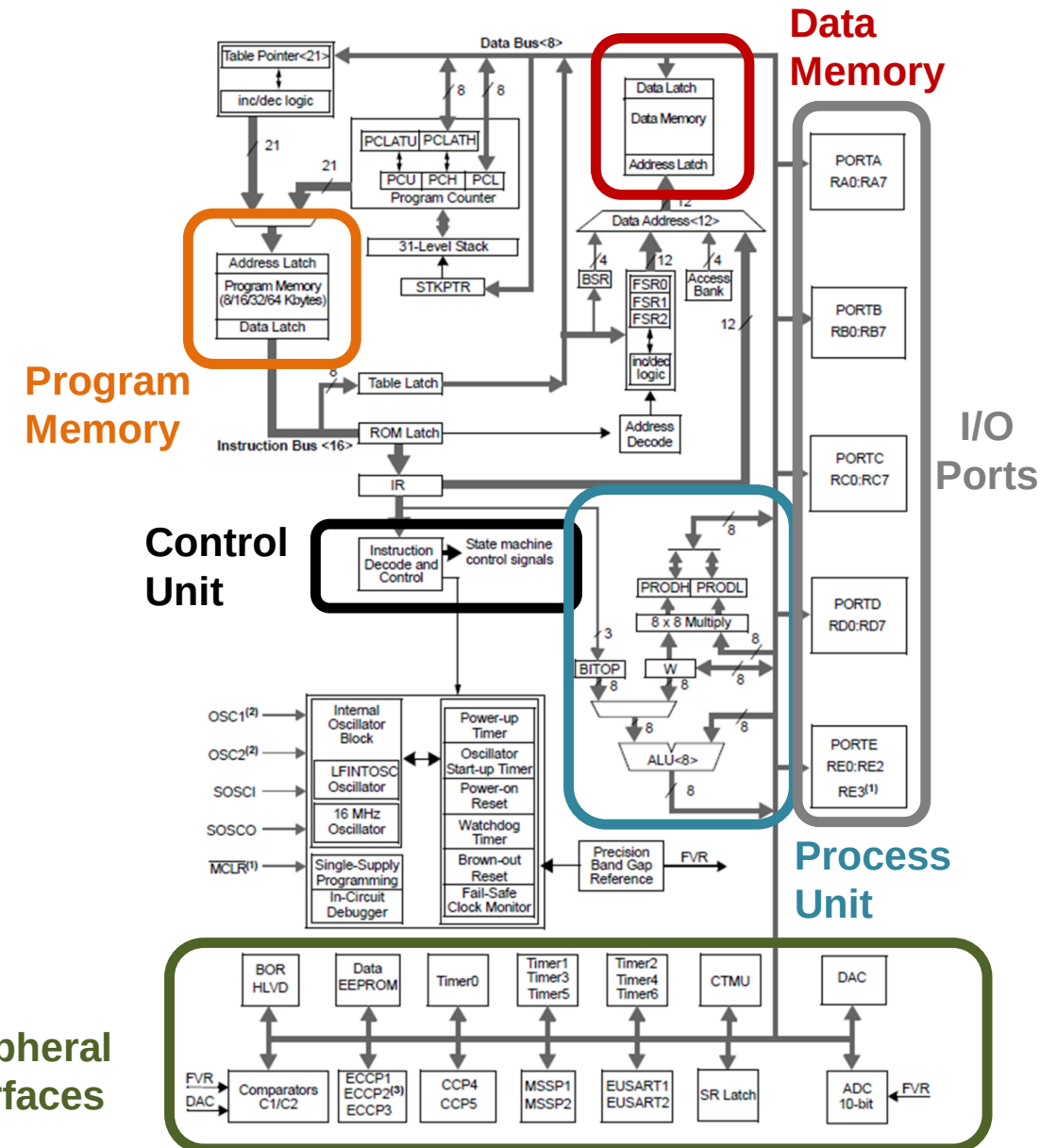
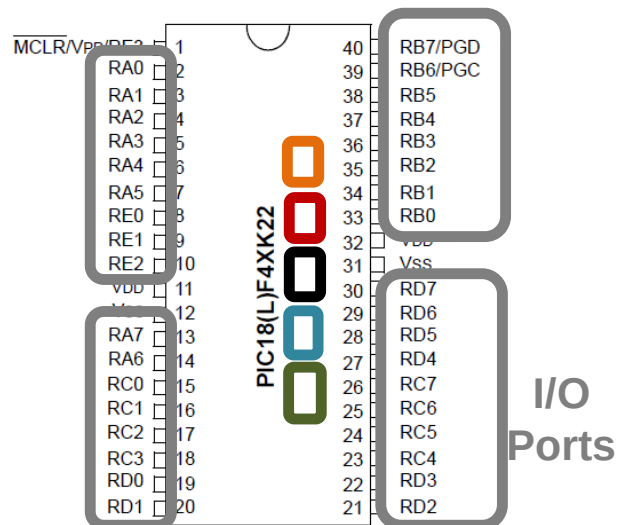
2.1 The 18F45K22 device

All the components (memories, CPU...) are inside the chip (is a microcontroller!).

I/O Ports group the physical pins (A, B, C...)

Besides, there are VDD, VSS (power) and MCLR (reset) pins.

40-PIN PDIP DIAGRAM



Peripheral Interfaces

2.1 The 18F45K22 device

A **family** (PIC18) has lots of **devices** to fit the requirements for every application.

Device description gives information about the exact Program memory and Data memory sizes, CPU features, Pin count, Clock speed, power requirements...

Features	PIC18F23K22 PIC18LF23K22	PIC18F24K22 PIC18LF24K22	PIC18F25K22 PIC18(L)F25K22	PIC18F26K22 PIC18LF26K22	PIC18F43K22 PIC18LF43K22	PIC18F44K22 PIC18LF44K22	PIC18F45K22 PIC18LF45K22
Program Memory (Bytes)	8192	16384	32768	65536	8192	16384	32768
Program Memory (Instructions)	4096	8192	16384	32768	4096	8192	16384
Data Memory (Bytes)	512	768	1536	3896	512	768	1536
Data EEPROM Memory (Bytes)	256	256	256	1024	256	256	256
I/O Ports	A, B, C, E ⁽¹⁾	A, B, C, E ⁽¹⁾	A, B, C, E ⁽¹⁾	A, B, C, E ⁽¹⁾	A, B, C, D, E	A, B, C, D, E	A, B, C, D, E
Capture/Compare/PWM Modules (CCP)	2	2	2	2	2	2	2
Enhanced CCP Modules (ECCP) - Half Bridge	2	2	2	2	1	1	1
Enhanced CCP Modules (ECCP) - Full Bridge	1	1	1	1	2	2	2
10-bit Analog-to-Digital Module (ADC)	2 Internal 17 Input	2 Internal 17 Input	2 Internal 17 Input	2 Internal 17 Input	2 Internal 28 Input	2 Internal 28 Input	2 Internal 28 Input
Packages	28-pin PDIP 28-pin SOIC 26-pin SSOP 28-pin QFN 28-pin UQFN	28-pin PDIP 28-pin SOIC 26-pin SSOP 28-pin QFN 28-pin UQFN	28-pin PDIP 28-pin SOIC 26-pin SSOP 28-pin QFN	28-pin PDIP 28-pin SOIC 26-pin SSOP 28-pin QFN	40-pin PDIP 40-pin UQFN 44-pin QFN 44-pin TQFP	40-pin PDIP 40-pin UQFN 44-pin QFN 44-pin TQFP	40-pin PDIP 40-pin UQFN 44-pin QFN 44-pin TQFP
Interrupt Sources				33			
Timers (16-bit)				4			
Serial Communications				2 MSSP, 2 EUSART			
SR Latch				Yes			
Charge Time Measurement Unit Module (CTMU)				Yes			
Programmable High/Low-Voltage Detect (HLVD)				Yes			
Programmable Brown-out Reset (BOR)				Yes			
Resets (and Delays)				POR, BOR, RESET Instruction, Stack Overflow, Stack Underflow (PWRT, OST), MCLR, WDT			
Instruction Set				75 Instructions; 83 with Extended Instruction Set enabled			
Operating Frequency				DC - 64 MHz			

2.1 The 18F45K22 device

Program Memory Bus:

21 bit address bus

16 bit wide instruction

(32KB out of 2MB max)

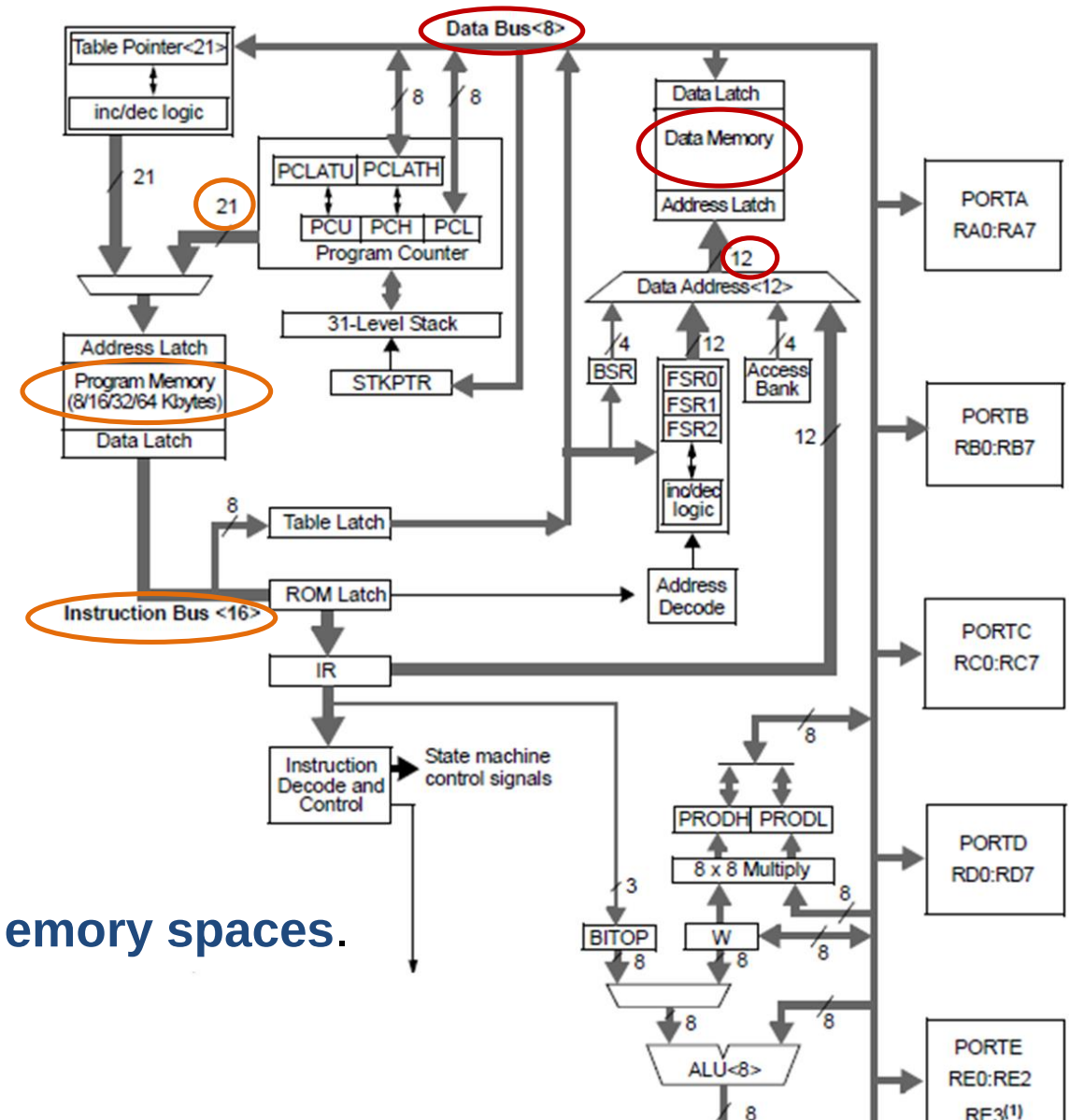
Data Memory Bus:

12 bit address bus

8 bit wide data

(1536B out of 4096B max)

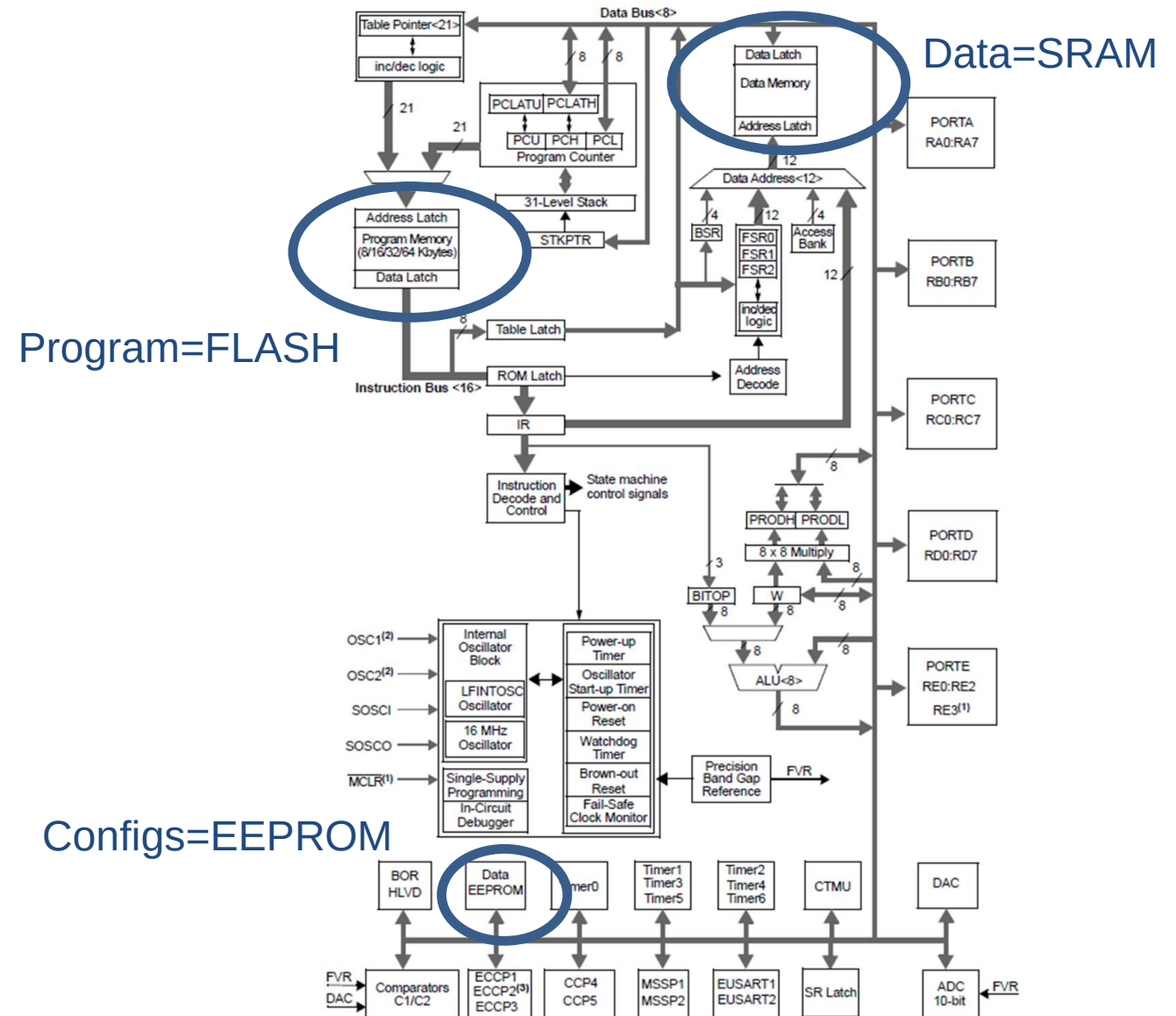
Separation of **Data Memory** and **Program Memory** spaces.
Remember: Harvard Architecture!



2.1 The 18F45K22 device

Memory technologies on PIC18 family.

- Volatile SRAM for Data
- FLASH memory for Code.
- Rewritable EEPROM for USER/APP configurations (language, data formats...)



2.1 The 18F45K22 device

Other characteristics

Stack:

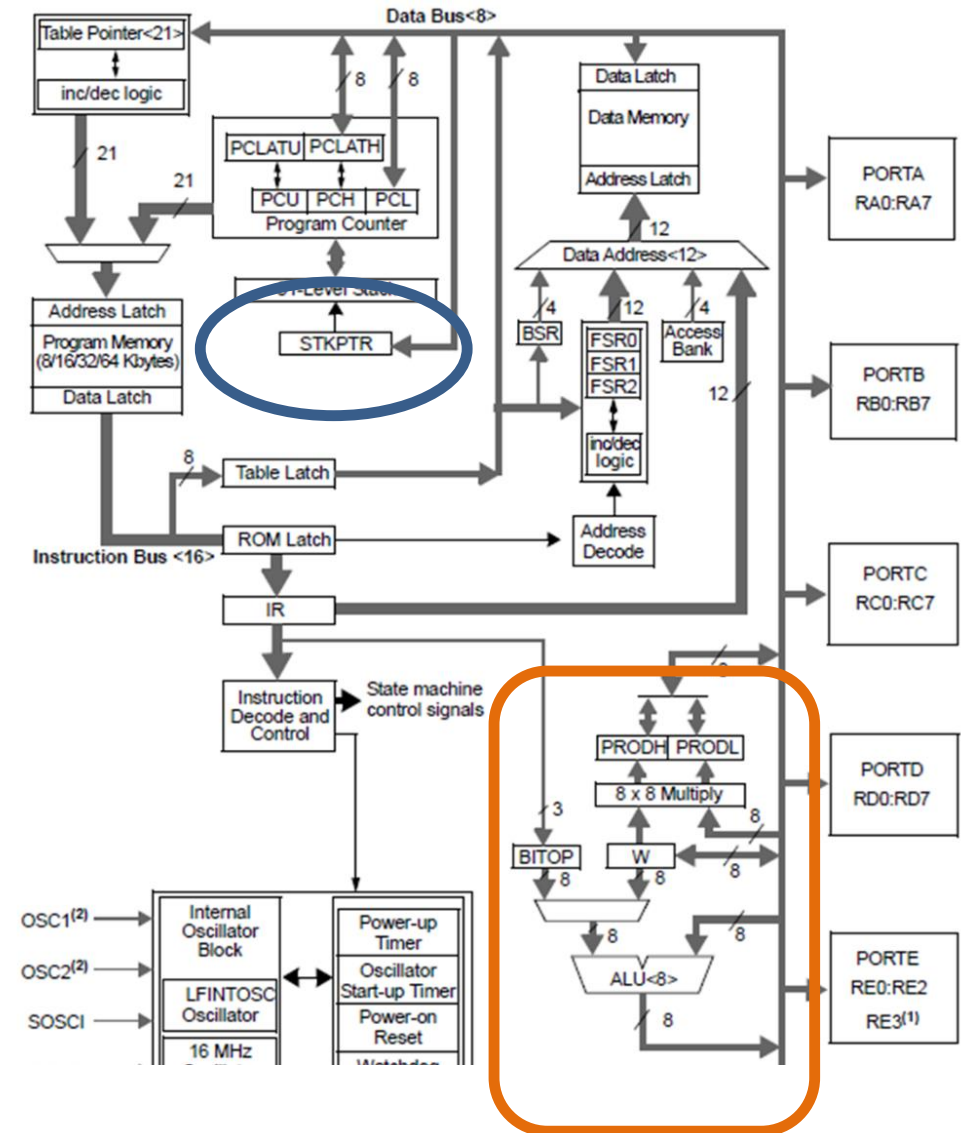
31-level Stack with STKPTR register.

Process Unit:

8 bit ALU

W Register, near ALU

8x8 bit multiplier hardware, result into specific registers.



2.2 Program memory organization

The **program counter (PC) is 21-bit long**, which enables the user program to access up to 2 MB of program memory (This features applies to PIC18 family).

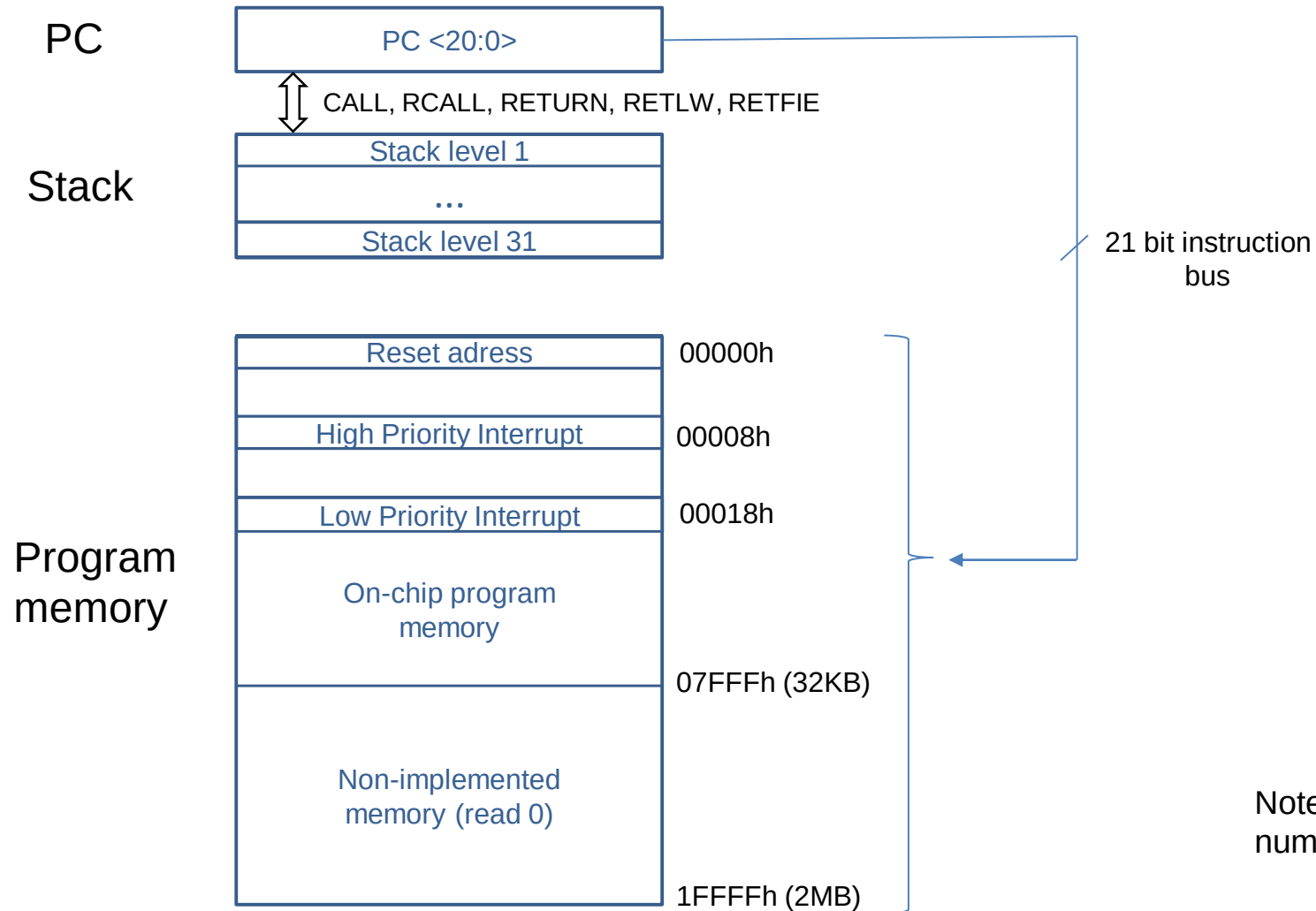
Up to 32KB of program memory is inside the MCU chip in the PIC18F45K22 device. (Accessing a location **beyond the upper boundary** of the physically implemented memory will **return all '0's**)

The PIC18 has a 31-entry return address stack to hold the return address for subroutine call.

- After power-on (or reset), the PIC18 starts to execute instructions from address 0 (00000h).
- The location at address 00008h is reserved for high-priority interrupt service routine.
- The location at address 00018h is reserved for low-priority interrupt service routine.

Almost all **instructions** in program memory are stored (opcode + parameters) in **16 bits / 2 bytes / 1 word**.

2.2 Program memory organization



Note: 00018h means that the number is coded hexadecimal.

2.2 Program memory organization

Instruction coding in program memory

The program memory is addressed in bytes.

Instructions are stored as **two bytes (1 word)** or **four bytes (2 words)** in program memory.

The Least Significant Byte of an instruction word is always stored in a program memory location with an even address (LSb = 0).

To align with instruction boundaries, the PC increments in steps of 2 and the LSb will always read '0'.

			Word Address	
			LSb = 1	LSb = 0
			↓	
Program Memory Byte Locations →				000000h
				000002h
				000004h
				000006h
Instruction 1:	MOVLW	055h	0Eh	55h
Instruction 2:	GOTO	0006h	EFh	03h
Instruction 3:	MOVFF	123h, 456h	F0h	00h
			C1h	23h
			F4h	56h

2.2 Program memory organization

The standard PIC18 instruction set has only four two-word instructions:
CALL, MOVFF, GOTO and LFSR.

In all cases, the second word of the instructions always has '1111' as its four Most Significant bits.
If the instruction is executed in proper sequence, immediately after the first word, the data in the second word is accessed and used by the instruction sequence. If the first word is **skipped** for some reason and the second word is executed by itself, a **NOP** is executed instead

CASE 1:	
Object Code	Source Code
0110 0110 0000 0000	TSTFSZ REG1 ; is RAM location 0?
1100 0001 0010 0011	MOVFF REG1, REG2 ; Yes, skip this word
1111 0100 0101 0110	; Execute this word as a NOP
0010 0100 0000 0000	ADDWF REG3 ; continue code
CASE 2:	
Object Code	Source Code
0110 0110 0000 0000	TSTFSZ REG1 ; is RAM location 0?
1100 0001 0010 0011	MOVFF REG1, REG2 ; No, execute this word
1111 0100 0101 0110	; 2nd word of instruction
0010 0100 0000 0000	ADDWF REG3 ; continue code

2.3 Data memory organization

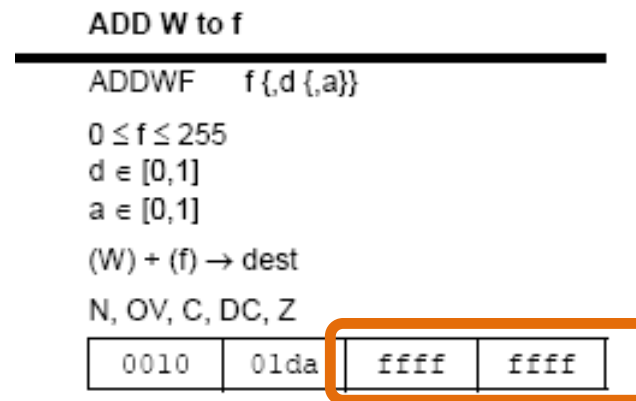
Implemented in SRAM and consists of **general-purpose registers** (GPR) and **special-function registers** (SFR). Both are referred to as data registers.

General-purpose registers (GPR) are used to hold dynamic data (e.g. variables).

Special-function registers (SFR) are used to control the operation of peripheral functions.

Most instructions use 8 bits to specify a data register (f field).

Eight bits can specify only 256 registers. This limitation forces the PIC18 to divide data registers into **banks**.



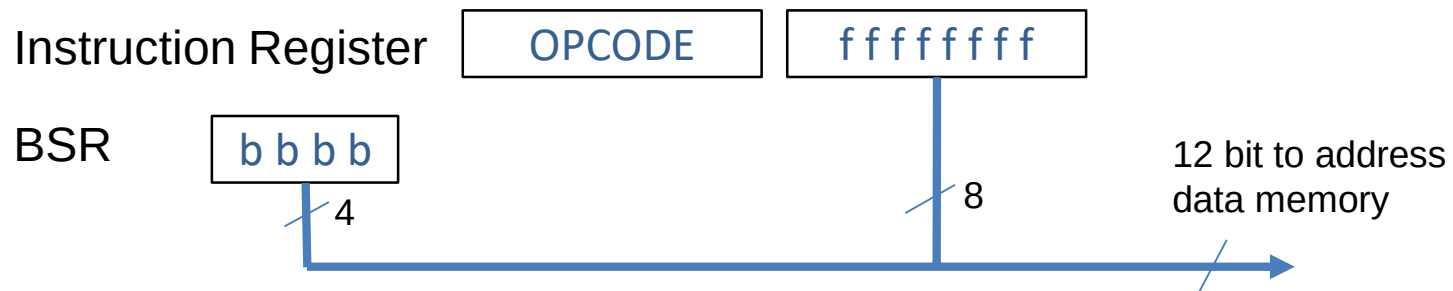
Add W to register 'f'. If 'd' is '0', the result is stored in W. If 'd' is '1', the result is stored back in register 'f' (default).
 If 'a' is '0', the Access Bank is selected. If 'a' is '1', the BSR is used to select the GPR bank (default).

2.3 Data memory organization

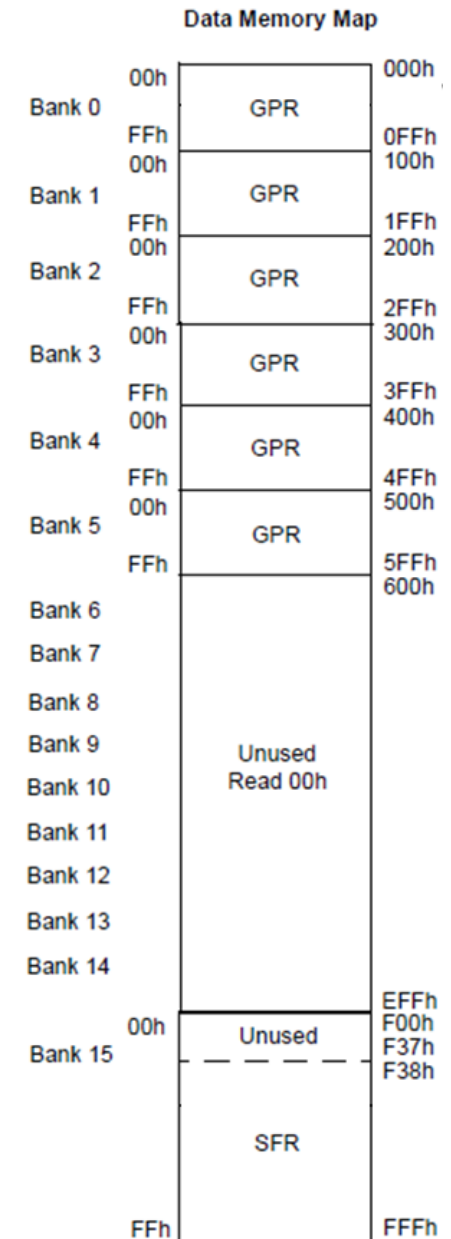
PIC18F45K22 implements seven banks:

- 6 **GPR**-based for a total of $6 \times 256\text{B} = 1,5\text{KB}$
- 1 **SFR**-based (200B implemented out of 256 possible)

A special register: BSR (Bank Select Register) is used to choose which Bank are we working with. It provides the 4 complementary bits to address Data Memory.



(The BSR can be loaded directly by using the **MOVLB instruction**)



2.3 Data memory organization

Banked mode

Only one bank is active at a time (the one specified by the BSR register)

When operating on a data register in a different Bank, **bank switching** is needed.

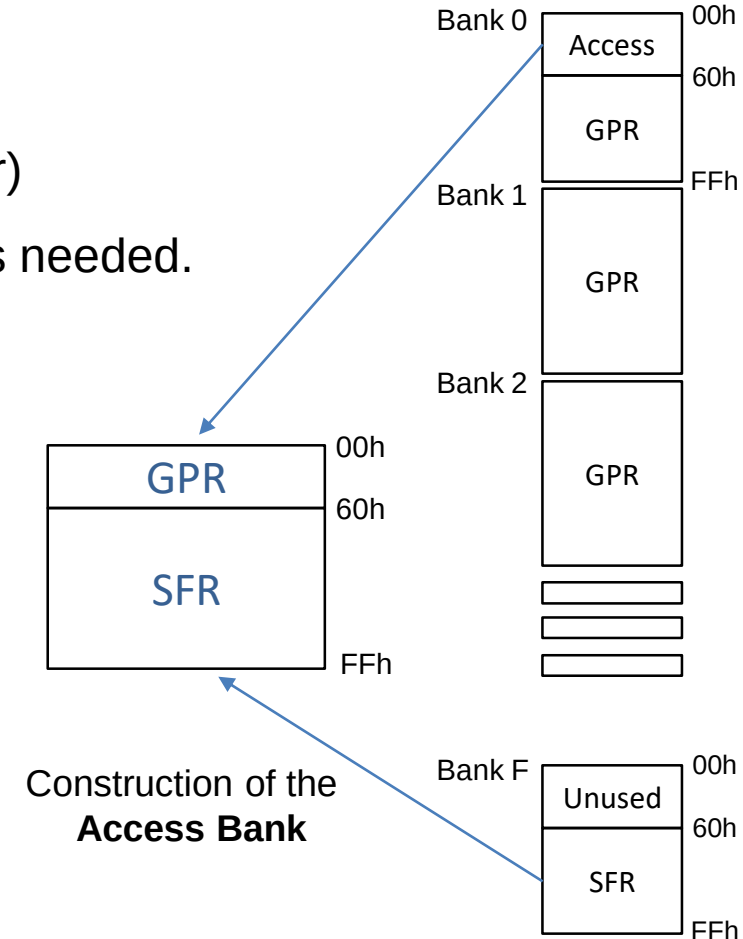
Bank switching incurs overhead and may cause program errors.

Access mode

Access bank is created to minimize the problems of bank switching.

Access bank consists of the lowest 96 bytes in general-purpose registers (from Bank 0) and the highest 160 bytes of special function registers (from Bank 15).

Short programs with few variables needed (to be stored in GPR) can use access mode and avoid Bank Switching.



2.3 Data memory organization

Use of Access or Banked mode

To indicate use of Access Bank or BSR we use the 'a' parameter in many instructions. Let's see an example with movwf (move working to file) instruction:

movwf f, a

When 'a' = 0: *The BSR is ignored and the Access Bank is used.*

When 'a' = 1: *The BSR specifies the bank used by the instruction*

The d (destiny) parameter

Most instructions have a 'd' parameter. Let's see an example with addwf (add working with file) instruction:

addwf f, d, a

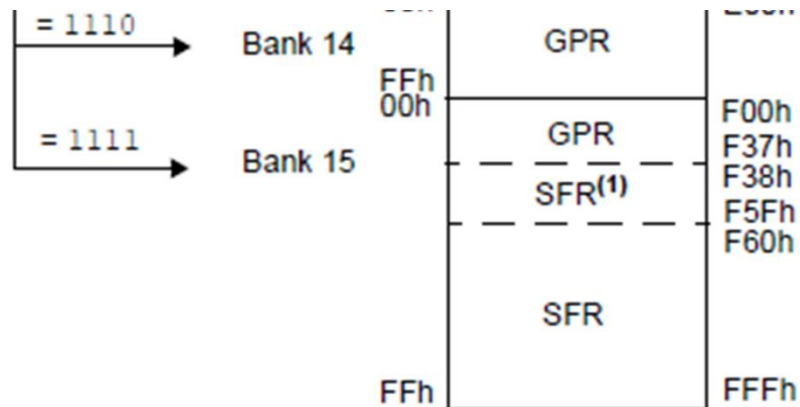
When 'd' = 0: *The result of the addition will be the Working Register.*

When 'd' = 1: *The result of the addition will be the File specified by f.*

2.3 Data memory organization

Special Function Registers

The group of registers from 0xF38 to 0xFFFF are dedicated to the general control of MCU operation and peripherals. Only registers placed between 0xF60 and 0xFFFF are mapped in the access bank.



Note 1: Addresses F38h through F5Fh are also used by SFRs, but are not part of the Access RAM. Users must always use the complete address or load the proper BSR value to access these registers.

The **WREG** register is involved in the execution of many instructions.

The **STATUS** register holds the status flags for the instruction execution .

2.3 Data memory organization

STATUS REGISTER

Contains bits related with the result of operations in the ALU.

REGISTER 5-2: STATUS REGISTER

U-0	U-0	U-0	R/W-x	R/W-x	R/W-x	R/W-x	R/W-x
—	—	—	N	OV	Z	DC ⁽¹⁾	C ⁽²⁾
bit 7			bit 0				

Legend:

R = Readable bit
-n = Value at POR

W = Writable bit
'1' = Bit is set

U = Unimplemented bit, read as '0'
'0' = Bit is cleared

x = Bit is unknown

- bit 4 N: Negative bit**
 This bit is used for signed arithmetic (2's complement). It indicates whether the result was negative (ALU MSB = 1).
 1 = Result was negative
 0 = Result was positive
- bit 3 OV: Overflow bit**
 This bit is used for signed arithmetic (2's complement). It indicates an overflow of the 7-bit magnitude which causes the sign bit (bit 7 of the result) to change state.
 1 = Overflow occurred for signed arithmetic (in this arithmetic operation)
 0 = No overflow occurred
- bit 2 Z: Zero bit**
 1 = The result of an arithmetic or logic operation is zero
 0 = The result of an arithmetic or logic operation is not zero
- bit 1 DC: Digit Carry/Borrow bit⁽¹⁾**
 For ADDWF, ADDLW, SUBLW and SUBWF instructions:
 1 = A carry-out from the 4th low-order bit of the result occurred
 0 = No carry-out from the 4th low-order bit of the result
- bit 0 C: Carry/Borrow bit⁽²⁾**
 For ADDWF, ADDLW, SUBLW and SUBWF instructions:
 1 = A carry-out from the Most Significant bit of the result occurred
 0 = No carry-out from the Most Significant bit of the result occurred

2.3 Data memory organization

The most common-use registers are mapped in the access bank.

SFR mapping in access Bank

but... ANSELA, ANSELB, ANSELC, ANSELD and ANSELE, needed to configure the pins as analog or digital, are not.

Note

- 1:** Not a physical register.
- 2:** Unimplemented registers are read as '0'.
- 3:** These registers are implemented only on 40/44-pin devices.

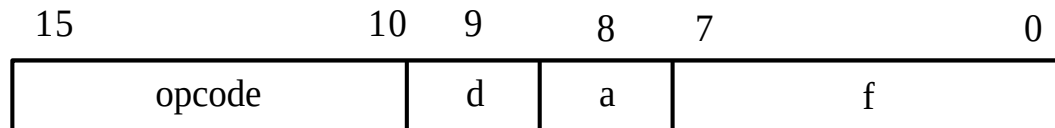
TABLE 5-1: SPECIAL FUNCTION REGISTER MAP FOR PIC18(L)F2X/4XK22 DEVICES

Address	Name	Address	Name	Address	Name	Address	Name	Address	Name
FFFh	TOSU	FD7h	TMR0H	FAFh	SPBRG1	F87h	— ⁽²⁾	F5Fh	CCPR3H
FFEh	TOSH	FD6h	TMR0L	FAEh	RCREG1	F86h	— ⁽²⁾	F5Eh	CCPR3L
FFDh	TOSL	FD5h	T0CON	FADh	TXREG1	F85h	— ⁽²⁾	F5Dh	CCP3CON
FFCh	STKPTR	FD4h	— ⁽²⁾	FACH	TXSTA1	F84h	PORTE	F5Ch	PWM3CON
FFBh	PCLATU	FD3h	OSCCON	FABh	RCSTA1	F83h	PORTD ⁽³⁾	F5Bh	ECCP3AS
FFAh	PCLATH	FD2h	OSCCON2	FAAh	EEADRH ⁽⁴⁾	F82h	PORTC	F5Ah	PSTR3CON
FF9h	PCL	FD1h	WDTCN	FA9h	EEADR	F81h	PORTB	F59h	CCPR4H
FF8h	TBLPTRU	FD0h	RCON	FA8h	EEDATA	F80h	PORTA	F58h	CCPR4L
FF7h	TBLPTRH	FCFh	TMR1H	FA7h	EECON2 ⁽¹⁾	F7Fh	IPR5	F57h	CCP4CON
FF6h	TBLPTRL	FCEh	TMR1L	FA6h	EECON1	F7Eh	PIR5	F56h	CCPR5H
FF5h	TABLAT	FCDh	T1CON	FA5h	IPR3	F7Dh	PIE5	F55h	CCPR5L
FF4h	PRODH	FCCh	T1GCON	FA4h	PIR3	F7Ch	IPR4	F54h	CCP5CON
FF3h	PRODL	FCBh	SSP1CON3	FA3h	PIE3	F7Bh	PIR4	F53h	TMR4
FF2h	INTCON	FCAh	SSP1MSK	FA2h	IPR2	F7Ah	PIE4	F52h	PR4
FF1h	INTCON2	FC9h	SSP1BUF	FA1h	PIR2	F79h	CM1CON0	F51h	T4CON
FF0h	INTCON3	FC8h	SSP1ADD	FA0h	PIE2	F78h	CM2CON0	F50h	TMR5H
FEFh	INDF0 ⁽¹⁾	FC7h	SSP1STAT	F9Fh	IPR1	F77h	CM2CON1	F4Fh	TMR5L
FEeh	POSTINC0 ⁽¹⁾	FC6h	SSP1CON1	F9Eh	PIR1	F76h	SPBRGH2	F4Eh	T5CON
FEDh	POSTDEC0 ⁽¹⁾	FC5h	SSP1CON2	F9Dh	PIE1	F75h	SPBRG2	F4Dh	T5GCON
FECh	PREINC0 ⁽¹⁾	FC4h	ADRESH	F9Ch	HLVDCON	F74h	RCREG2	F4Ch	TMR6
FEbh	PLUSW0 ⁽¹⁾	FC3h	ADRESL	F9Bh	OSCTUNE	F73h	TXREG2	F4Bh	PR6
FEAh	FSR0H	FC2h	ADCON0	F9Ah	— ⁽²⁾	F72h	TXSTA2	F4Ah	T6CON
FE9h	FSR0L	FC1h	ADCON1	F99h	— ⁽²⁾	F71h	RCSTA2	F49h	CCPTMRS0
FE8h	WREG	FC0h	ADCON2	F98h	— ⁽²⁾	F70h	BAUDCON2	F48h	CCPTMRS1
FE7h	INDF1 ⁽¹⁾	FBFh	CCPR1H	F97h	— ⁽²⁾	F6Fh	SSP2BUF	F47h	SRCON0
FE6h	POSTINC1 ⁽¹⁾	FBEh	CCPR1L	F96h	TRISE	F6Eh	SSP2ADD	F46h	SRCON1
FE5h	POSTDEC1 ⁽¹⁾	FBDh	CCP1CON	F95h	TRISD ⁽³⁾	F6Dh	SSP2STAT	F45h	CTMUCONH
FE4h	PREINC1 ⁽¹⁾	FBCh	TMR2	F94h	TRISC	F6Ch	SSP2CON1	F44h	CTMUCONL
FE3h	PLUSW1 ⁽¹⁾	FBBh	PR2	F93h	TRISB	F6Bh	SSP2CON2	F43h	CTMUICON
FE2h	FSR1H	FBAh	T2CON	F92h	TRISA	F6Ah	SSP2MSK	F42h	VREFCON0
FE1h	FSR1L	FB9h	PSTR1CON	F91h	— ⁽²⁾	F69h	SSP2CON3	F41h	VREFCON1
FE0h	BSR	FB8h	BAUDCON1	F90h	— ⁽²⁾	F68h	CCPR2H	F40h	VREFCON2
FDFh	INDF2 ⁽¹⁾	FB7h	PWM1CON	F8Fh	— ⁽²⁾	F67h	CCPR2L	F3Fh	PMD0
FDEh	POSTINC2 ⁽¹⁾	FB6h	ECCP1AS	F8Eh	— ⁽²⁾	F66h	CCP2CON	F3Eh	PMD1
FDDh	POSTDEC2 ⁽¹⁾	FB5h	— ⁽²⁾	F8Dh	LATE ⁽³⁾	F65h	PWM2CON	F3Dh	PMD2
FDCh	PREINC2 ⁽¹⁾	FB4h	T3GCON	F8Ch	LATD ⁽³⁾	F64h	ECCP2AS	F3Ch	ANSELE
FDBh	PLUSW2 ⁽¹⁾	FB3h	TMR3H	F8Bh	LATC	F63h	PSTR2CON	F3Bh	ANSELD
FDAh	FSR2H	FB2h	TMR3L	F8Ah	LATB	F62h	IOCB	F3Ah	ANSELC
FD9h	FSR2L	FB1h	T3CON	F89h	LATA	F61h	WPUB	F39h	ANSELB
FD8h	STATUS	FB0h	SPBRGH1	F88h	— ⁽²⁾	F60h	SLRCON	F38h	ANSELA

2.4 Instruction set

Five instruction types:

Byte-oriented operations



d = 0 for result destination to be WREG register.

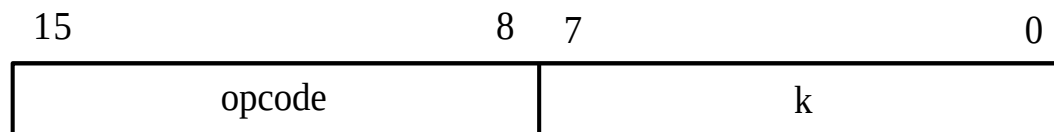
d = 1 for result destination to be file register (f)

a = 0 to force Access Bank

a = 1 for BSR to select bank

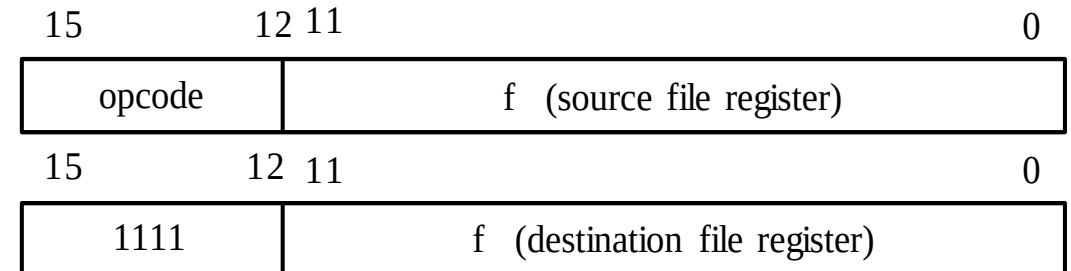
f = 8-bit file register address

Literal operations



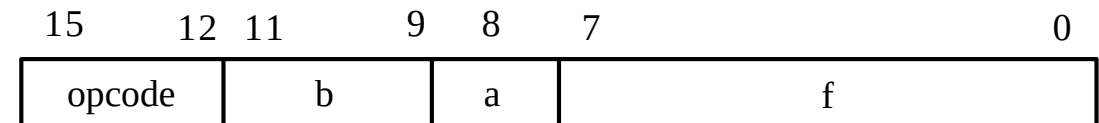
k = 8-bit immediate value

Byte-to-byte move operations (two words)



f = 12-bit file register address

Bit-oriented file register operations



b = 3-bit position of bit in the file register (f).

a = 0 to force Access Bank

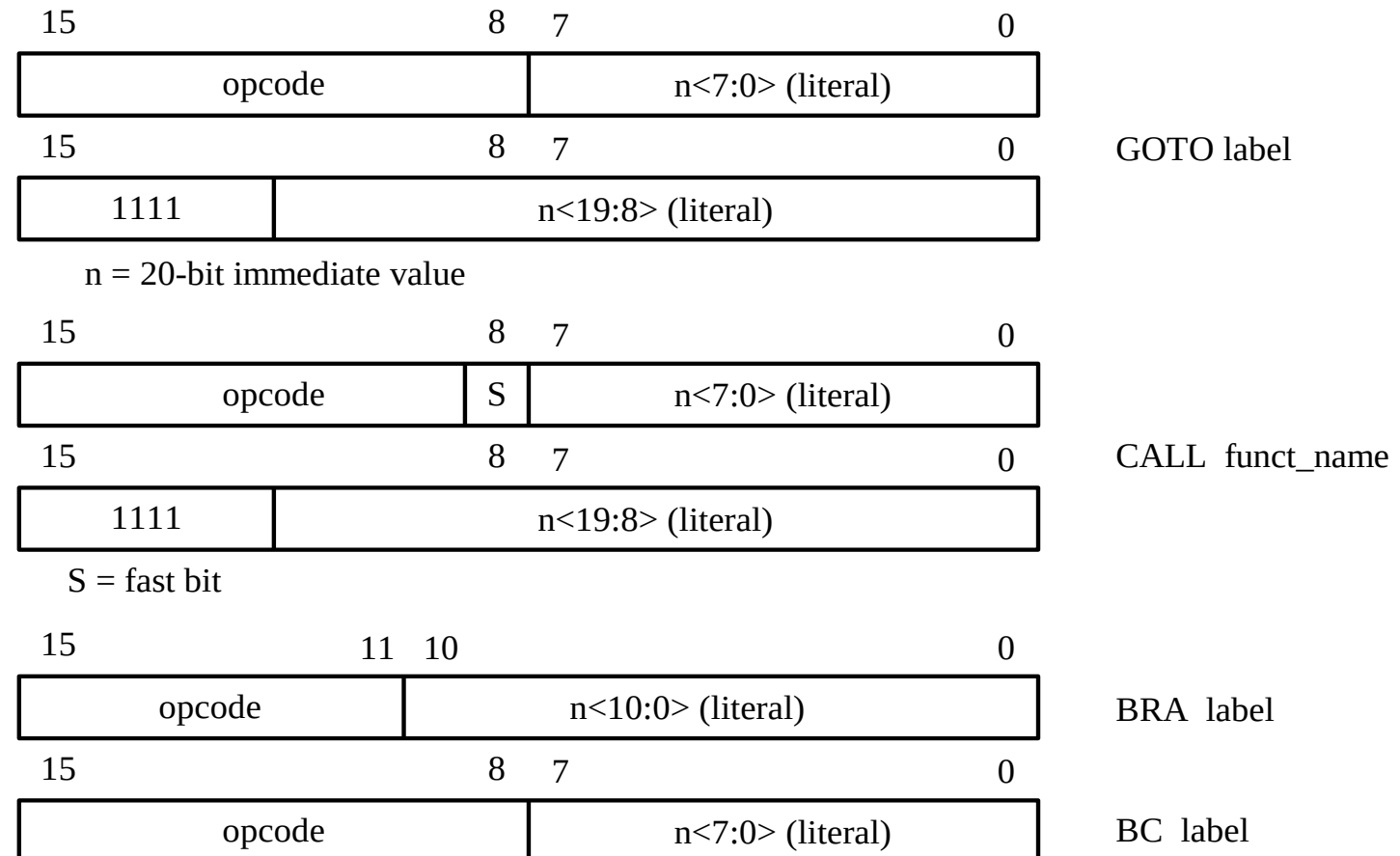
a = 1 for BSR to select bank

f = 8-bit file register address

2.4 Instruction set

Five instruction types (cont):

Control operations: used to change the program execution sequence and making subroutine calls



2.4 Instruction set

For every instruction we have:

- Mnemonic and operands
- Execution cycles
- Memory coding
- Status bits affected (if any)

TABLE 25-2: PIC18(L)F2X/4XK22 INSTRUCTION SET

Mnemonic, Operands	Description	Cycles	16-Bit Instruction Word				Status Affected	Notes	
			MSb		LSb				
BYTE-ORIENTED OPERATIONS									
ADDWF	f, d, a	Add WREG and f	1	0010	01da	ffff	ffff	C, DC, Z, OV, N	1, 2
ADDWFC	f, d, a	Add WREG and CARRY bit to f	1	0010	00da	ffff	ffff	C, DC, Z, OV, N	1, 2
ANDWF	f, d, a	AND WREG with f	1	0001	01da	ffff	ffff	Z, N	1, 2
CLRF	f, a	Clear f	1	0110	101a	ffff	ffff	Z	2
COMF	f, d, a	Complement f	1	0001	11da	ffff	ffff	Z, N	1, 2
CPFSEQ	f, a	Compare f with WREG, skip =	1 (2 or 3)	0110	001a	ffff	ffff	None	4
CPFSGT	f, a	Compare f with WREG, skip >	1 (2 or 3)	0110	010a	ffff	ffff	None	4
CPFSLT	f, a	Compare f with WREG, skip <	1 (2 or 3)	0110	000a	ffff	ffff	None	1, 2
DECF	f, d, a	Decrement f	1	0000	01da	ffff	ffff	C, DC, Z, OV, N	1, 2, 3, 4
DECFSZ	f, d, a	Decrement f, Skip if 0	1 (2 or 3)	0010	11da	ffff	ffff	None	1, 2, 3, 4
DCFSNZ	f, d, a	Decrement f, Skip if Not 0	1 (2 or 3)	0100	11da	ffff	ffff	None	1, 2
INCF	f, d, a	Increment f	1	0010	10da	ffff	ffff	C, DC, Z, OV, N	1, 2, 3, 4
INCFSZ	f, d, a	Increment f, Skip if 0	1 (2 or 3)	0011	11da	ffff	ffff	None	4
INFSNZ	f, d, a	Increment f, Skip if Not 0	1 (2 or 3)	0100	10da	ffff	ffff	None	1, 2
IORWF	f, d, a	Inclusive OR WREG with f	1	0001	00da	ffff	ffff	Z, N	1, 2
MOVF	f, d, a	Move f	1	0101	00da	ffff	ffff	Z, N	1
MOVFF	f _s , f _d	Move f _s (source) to f _d (destination)	2	1100	ffff	ffff	ffff	None	
		1st word		1111	ffff	ffff	ffff		
MOVWF	f, a	Move WREG to f	1	0110	111a	ffff	ffff	None	
MULWF	f, a	Multiply WREG with f	1	0000	001a	ffff	ffff	None	1, 2
NEGF	f, a	Negate f	1	0110	110a	ffff	ffff	C, DC, Z, OV, N	
RLCF	f, d, a	Rotate Left f through Carry	1	0011	01da	ffff	ffff	C, Z, N	1, 2
RLNCF	f, d, a	Rotate Left f (No Carry)	1	0100	01da	ffff	ffff	Z, N	
RRCF	f, d, a	Rotate Right f through Carry	1	0011	00da	ffff	ffff	C, Z, N	
RRNCF	f, d, a	Rotate Right f (No Carry)	1	0100	00da	ffff	ffff	Z, N	
SETF	f, a	Set f	1	0110	100a	ffff	ffff	None	1, 2
SUBFWB	f, d, a	Subtract f from WREG with borrow	1	0101	01da	ffff	ffff	C, DC, Z, OV, N	
SUBWF	f, d, a	Subtract WREG from f	1	0101	11da	ffff	ffff	C, DC, Z, OV, N	1, 2
SUBWFB	f, d, a	Subtract WREG from f with borrow	1	0101	10da	ffff	ffff	C, DC, Z, OV, N	
SWAPF	f, d, a	Swap nibbles in f	1	0011	10da	ffff	ffff	None	4
TSTFSZ	f, a	Test f, skip if 0	1 (2 or 3)	0110	011a	ffff	ffff	None	1, 2
XORWF	f, d, a	Exclusive OR WREG with f	1	0001	10da	ffff	ffff	Z, N	

2.4 Instruction set

(2/3)

TABLE 25-2: PIC18(L)F2X/4XK22 INSTRUCTION SET (CONTINUED)

Mnemonic, Operands	Description	Cycles	16-Bit Instruction Word				Status Affected	Notes	
			MSb		LSb				
BIT-ORIENTED OPERATIONS									
BCF	f, b, a	Bit Clear f	1	1001	bbba	ffff	ffff	None	1, 2
BSF	f, b, a	Bit Set f	1	1000	bbba	ffff	ffff	None	1, 2
BTFSC	f, b, a	Bit Test f, Skip if Clear	1 (2 or 3)	1011	bbba	ffff	ffff	None	3, 4
BTFSS	f, b, a	Bit Test f, Skip if Set	1 (2 or 3)	1010	bbba	ffff	ffff	None	3, 4
BTG	f, b, a	Bit Toggle f	1	0111	bbba	ffff	ffff	None	1, 2
CONTROL OPERATIONS									
BC	n	Branch if Carry	1 (2)	1110	0010	nnnn	nnnn	None	4
BN	n	Branch if Negative	1 (2)	1110	0110	nnnn	nnnn	None	
BNC	n	Branch if Not Carry	1 (2)	1110	0011	nnnn	nnnn	None	
BNN	n	Branch if Not Negative	1 (2)	1110	0111	nnnn	nnnn	None	
BNOV	n	Branch if Not Overflow	1 (2)	1110	0101	nnnn	nnnn	None	
BNZ	n	Branch if Not Zero	1 (2)	1110	0001	nnnn	nnnn	None	
BOV	n	Branch if Overflow	1 (2)	1110	0100	nnnn	nnnn	None	
BRA	n	Branch Unconditionally	2	1101	0nnn	nnnn	nnnn	None	
BZ	n	Branch if Zero	1 (2)	1110	0000	nnnn	nnnn	None	
CALL	k, s	Call subroutine 1st word	2	1110	110s	kkkk	kkkk	None	
		2nd word		1111	kkkk	kkkk	kkkk		
CLRWDT	—	Clear Watchdog Timer	1	0000	0000	0000	0100	$\overline{TO}$, $\overline{PD}$	
DAW	—	Decimal Adjust WREG	1	0000	0000	0000	0111	C	
GOTO	k	Go to address 1st word	2	1110	1111	kkkk	kkkk	None	
		2nd word		1111	kkkk	kkkk	kkkk		
NOP	—	No Operation	1	0000	0000	0000	0000	None	
NOP	—	No Operation	1	1111	xxxx	xxxx	xxxx	None	
POP	—	Pop top of return stack (TOS)	1	0000	0000	0000	0110	None	
PUSH	—	Push top of return stack (TOS)	1	0000	0000	0000	0101	None	
RCALL	n	Relative Call	2	1101	1nnn	nnnn	nnnn	None	
RESET		Software device Reset	1	0000	0000	1111	1111	All	
RETFIE	s	Return from interrupt enable	2	0000	0000	0001	000s	GIE/GIEH, PEIE/GIEL	
RETLW	k	Return with literal in WREG	2	0000	1100	kkkk	kkkk	None	
RETURN	s	Return from Subroutine	2	0000	0000	0001	001s	None	
SLEEP	—	Go into Standby mode	1	0000	0000	0000	0011	$\overline{TO}$, $\overline{PD}$	

2.4 Instruction set

(3/3)

TABLE 25-2: PIC18(L)F2X/4XK22 INSTRUCTION SET (CONTINUED)

Mnemonic, Operands	Description	Cycles	16-Bit Instruction Word				Status Affected	Notes		
			MSb		LSb					
LITERAL OPERATIONS										
ADDLW	k	Add literal and WREG	1	0000	1111	kkkk	kkkk	C, DC, Z, OV, N		
ANDLW	k	AND literal with WREG	1	0000	1011	kkkk	kkkk	Z, N		
IORLW	k	Inclusive OR literal with WREG	1	0000	1001	kkkk	kkkk	Z, N		
LFSR	f, k	Move literal (12-bit) 2nd word to FSR(f) 1st word	2	1110	1110	00ff	kkkk	None		
MOVLB	k	Move literal to BSR<3:0>	1	0000	0001	0000	kkkk	None		
MOVLW	k	Move literal to WREG	1	0000	1110	kkkk	kkkk	None		
MULLW	k	Multiply literal with WREG	1	0000	1101	kkkk	kkkk	None		
RETLW	k	Return with literal in WREG	2	0000	1100	kkkk	kkkk	None		
SUBLW	k	Subtract WREG from literal	1	0000	1000	kkkk	kkkk	C, DC, Z, OV, N		
XORLW	k	Exclusive OR literal with WREG	1	0000	1010	kkkk	kkkk	Z, N		
DATA MEMORY ↔ PROGRAM MEMORY OPERATIONS										
TBLRD*	Table Read	2	0000	0000	0000	1000	None			
TBLRD*+	Table Read with post-increment	2	0000	0000	0000	1001	None			
TBLRD*-	Table Read with post-decrement		0000	0000	0000	1010	None			
TBLRD*+	Table Read with pre-increment		0000	0000	0000	1011	None			
TBLWT*	Table Write		0000	0000	0000	1100	None			
TBLWT*+	Table Write with post-increment		0000	0000	0000	1101	None			
TBLWT*-	Table Write with post-decrement		0000	0000	0000	1110	None			
TBLWT*+	Table Write with pre-increment		0000	0000	0000	1111	None			

- Note** 1: When a PORT register is modified as a function of itself (e.g., `MOVF PORTB, 1, 0`), the value used will be that value present on the pins themselves. For example, if the data latch is '1' for a pin configured as input and is driven low by an external device, the data will be written back with a '0'.
- 2: If this instruction is executed on the TMR0 register (and where applicable, 'd' = 1), the prescaler will be cleared if assigned.
- 3: If Program Counter (PC) is modified or a conditional test is true, the instruction requires two cycles. The second cycle is executed as a NOP.
- 4: Some instructions are two-word instructions. The second word of these instructions will be executed as a NOP unless the first word of the instruction retrieves the information embedded in these 16 bits. This ensures that all program memory locations have a valid instruction.

2.5 Addressing modes

The PIC18 family has **4 addressing modes**

- Inherent: don't need any argument

sleep, reset

- Literal: require an explicit argument in the opcode

movlw 0x30 ; load 0x30 into WREG

- Direct: specifies source and/or destination address

movf 0x20,d,a ; the value 0x20 is register direct mode

The destination bit ('d') can determine if the result is stored in memory or WREG.

The access bit ('a') determines if the 8-bit address (0x20) is in access or BSR Bank.

- Indirect: Access a location without giving a fixed address in the instruction.

The registers **FSR are used as pointers** (FSR not SFR!!).

2.5 Addressing modes

Indirect addressing

File Select Registers (**FSR_x**) are used as pointers to the actual data register.

- There are 3 FSR (x = 0,1,2)
- The registers for Indirect Addressing are also implemented with Indirect File Operands that permit automatic manipulation of the pointer value with auto-incrementing, auto-decrementing or offsetting with another value.
- 5 Indirect modes:

INDF	(just indexing)
POSTDEC	(use the pointer and decrement it)
POSTINC	(use the pointer and increment it)
PREINC	(increment the pointer and use it)
PLUSW	(use the pointer and add W to it)

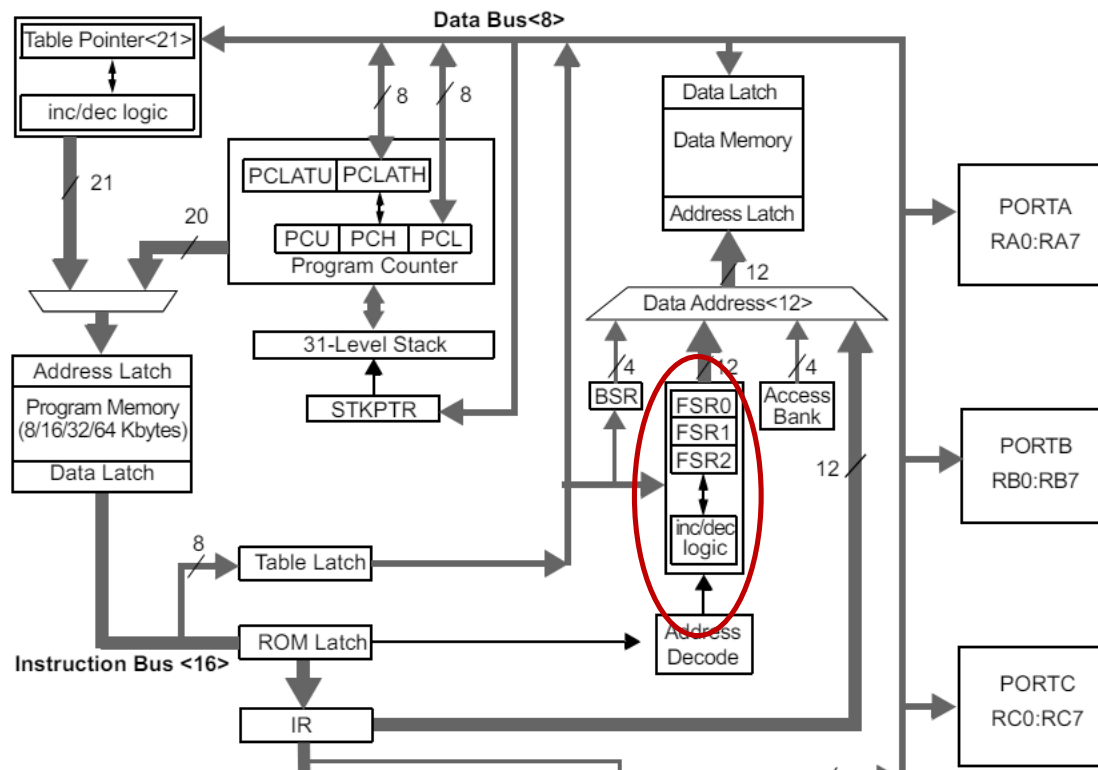
EXAMPLE 5-5: HOW TO CLEAR RAM (BANK 1) USING INDIRECT ADDRESSING

	LF SR	FSR0, 100h ;	
NEXT	CLRF	POSTINC0	; Clear indirect
			; register then
			; inc FSR pointer
	BTFSS	FSR0H, 1	; All done with
			; Bank1?
	BRA	NEXT	; NO, clear next
CONTINUE			; YES, continue

2.5 Addressing modes

Indirect addressing

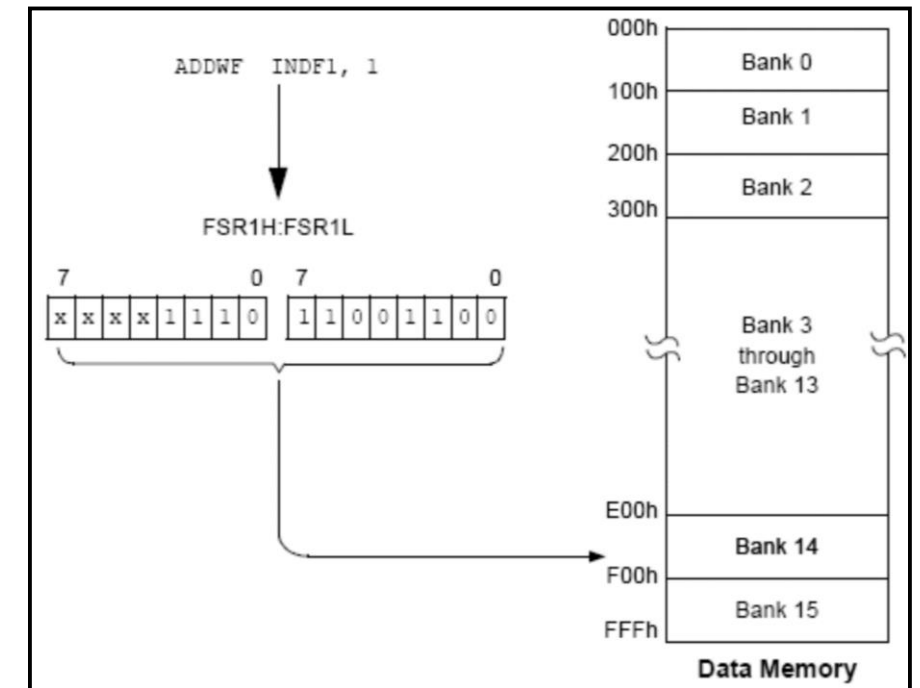
FSR in data bus control



FSR (H/L) generating 12 bit address

The 12-bit pointer is in two registers:

FSRxH (4 bits) and FSRxL (8 bits)

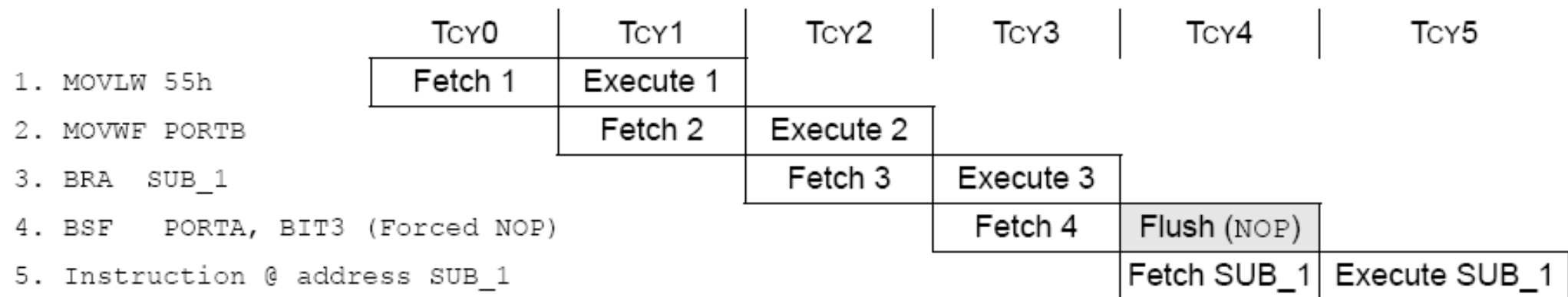


2.6 Pipelining and execution time

The PIC18 divide most of the instruction execution into two stages: **instruction fetch** and **instruction execution**.

Up to two instructions are overlapped in their execution.

One instruction is in fetch stage while the second instruction is in execution stage. Because of pipelining, each instruction **appears** to take one instruction cycle to complete.



2.6 Pipelining and execution time

All single-word instructions are executed in a single instruction cycle, unless a conditional test is true or the program counter is changed as a result of the instruction. In these cases, the execution takes two instruction cycles with the additional instruction cycle(s) executed as a NOP.

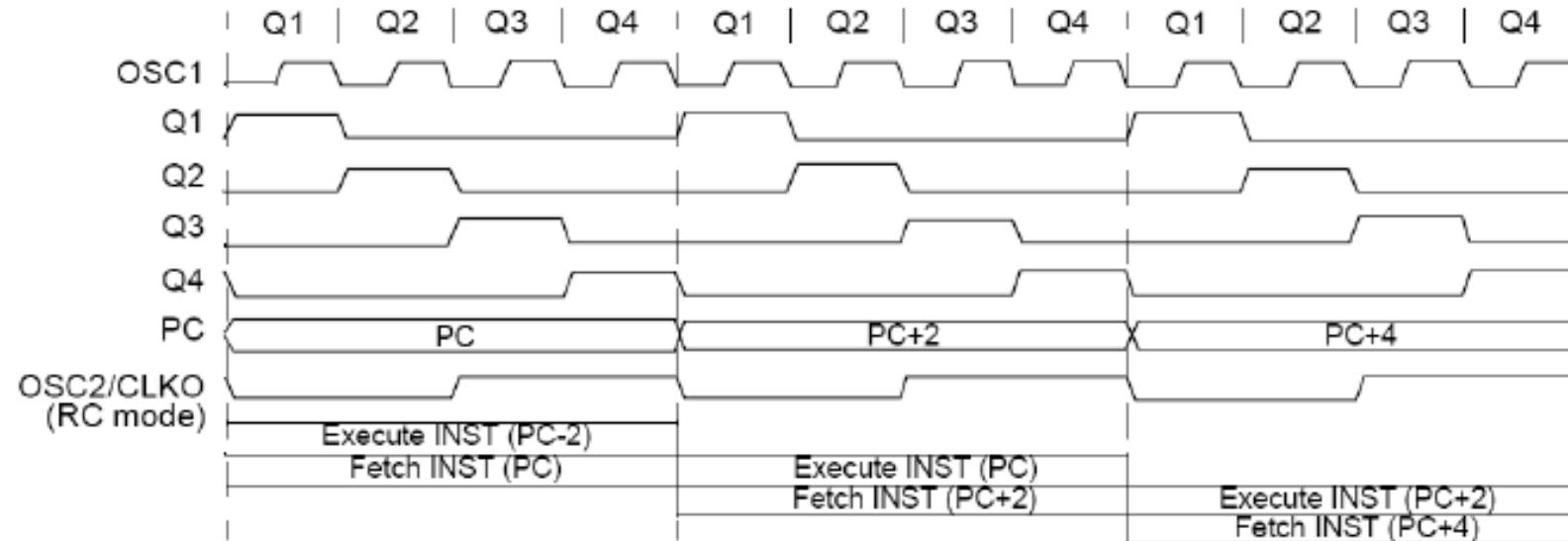
Branch instructions execute in two cycles. The double-word instructions execute in two instruction cycles.

Let's discuss some examples:

Mnemonic, Operands		Description	Cycles	16-Bit Instruction Word				Status Affected	Notes
				MSb		LSb			
BYTE-ORIENTED OPERATIONS									
ADDWF	f, d, a	Add WREG and f	1	0010	01da	ffff	ffff	C, DC, Z, OV, N	1, 2
ADDWFC	f, d, a	Add WREG and CARRY bit to f	1	0010	00da	ffff	ffff	C, DC, Z, OV, N	1, 2
CPFSEQ	f, a	Compare f with WREG, skip =	1 (2 or 3)	0110	001a	ffff	ffff	None	4
CPFSGT	f, a	Compare f with WREG, skip >	1 (2 or 3)	0110	010a	ffff	ffff	None	4
CPFSLT	f, a	Compare f with WREG, skip <	1 (2 or 3)	0110	000a	ffff	ffff	None	1, 2
BRA	n	Branch Unconditionally	2	1101	0nnn	nnnn	nnnn	None	
BZ	n	Branch if Zero	1 (2)	1110	0000	nnnn	nnnn	None	
MOVFF	f _s , f _d	Move f _s (source) to 1st word f _d (destination) 2nd word	2	1100	ffff	ffff	ffff	None	
				1111	ffff	ffff	ffff		

2.6 Pipelining and execution time

Every instruction cycle (fetch/exe) requires 4 system clocks.



Instruction subcycles (Q-cycles)

Q1. instruction decode subcycle

Q2. read data subcycle

Q3. process subcycle

Q4. write data subcycle

2.6 Pipelining and execution time

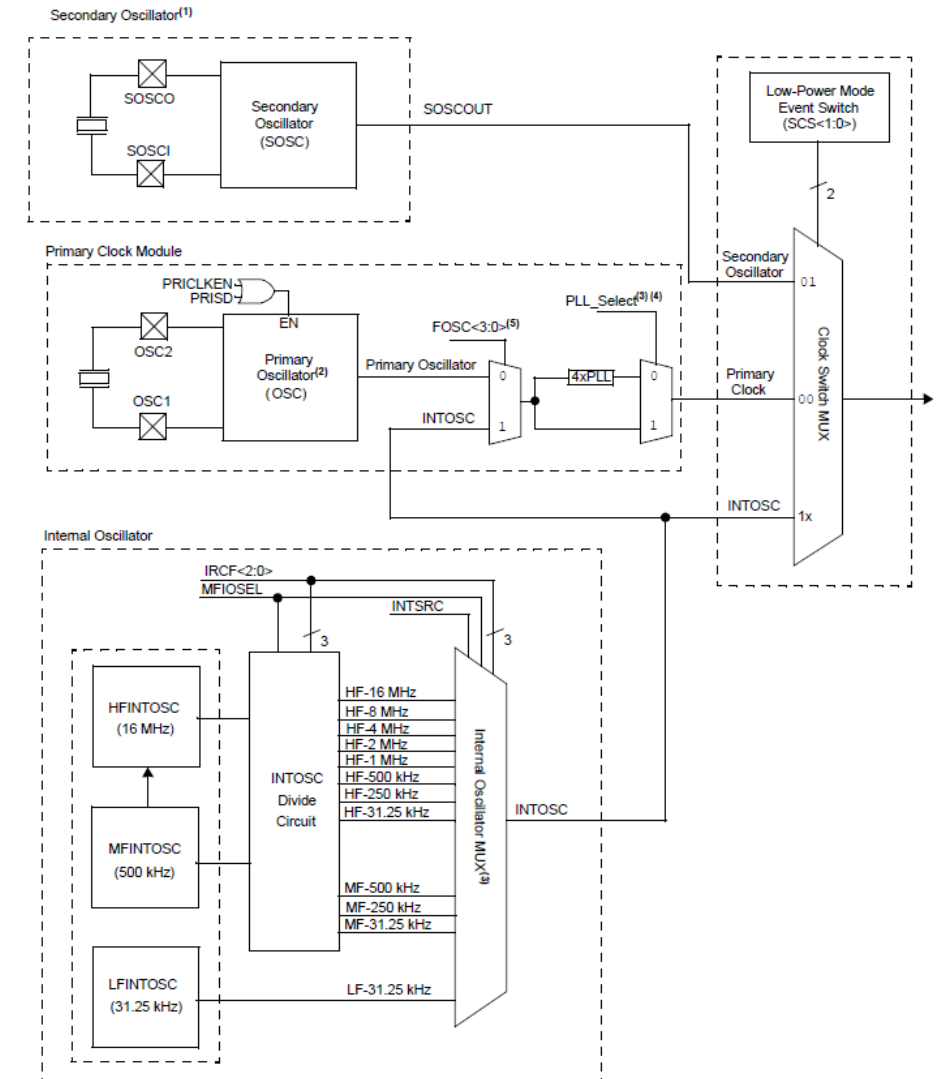
System clock is generated in the PIC1845K22 “system clock unit”.

External or internal oscillators and dividers are configured in order to choose our system clock.

PIC1845K22 in the **labs** is configured with a 8MHz system clock, so instruction cycle is 500ns.

$$f_{\text{OSC}} = 8 \text{ MHz} \rightarrow T_{\text{OSC}} = 125 \text{ ns} \rightarrow T_{\text{cyc}} = 500 \text{ ns}$$

$$f_{\text{OSC_MAX}} = 20 \text{ MHz} \rightarrow T_{\text{OSC}} = 50 \text{ ns} \rightarrow T_{\text{cyc}} = 200 \text{ ns}$$



2.6 Pipelining and execution time

Examples of instructions that change the program counter (breaking the pipeline):

BRA n: jump to the instruction with address equal to $PC+2+n$

GOTO n: jump to address represented by **n**

B_{CC} n: jump to the instruction with address equal to $PC+2+n$ if the condition code CC is true.

CC can be any one of the following:

C: if C (Carry) flag is set to 1

N: if N (Negative) flag is set to 1 which indicates that the previous operation result was negative

NC: if C flag is 0

NN: if N flag is 0 which indicates non-negative condition

OV: if OV flag is 1 indicating previous operation caused an overflow

Z: if Z flag is 1 which indicates the previous operation result was zero

NOV: if OV (Overflow) flag is 0 indicating there is no overflow condition

NZ: if Z (Zero) flag is 0 indicating previous operation result was not zero

CPFSEQ f,a: compare register f with WREG, skip if equal

INCFSZ f,d,a: increment f, skip if 0

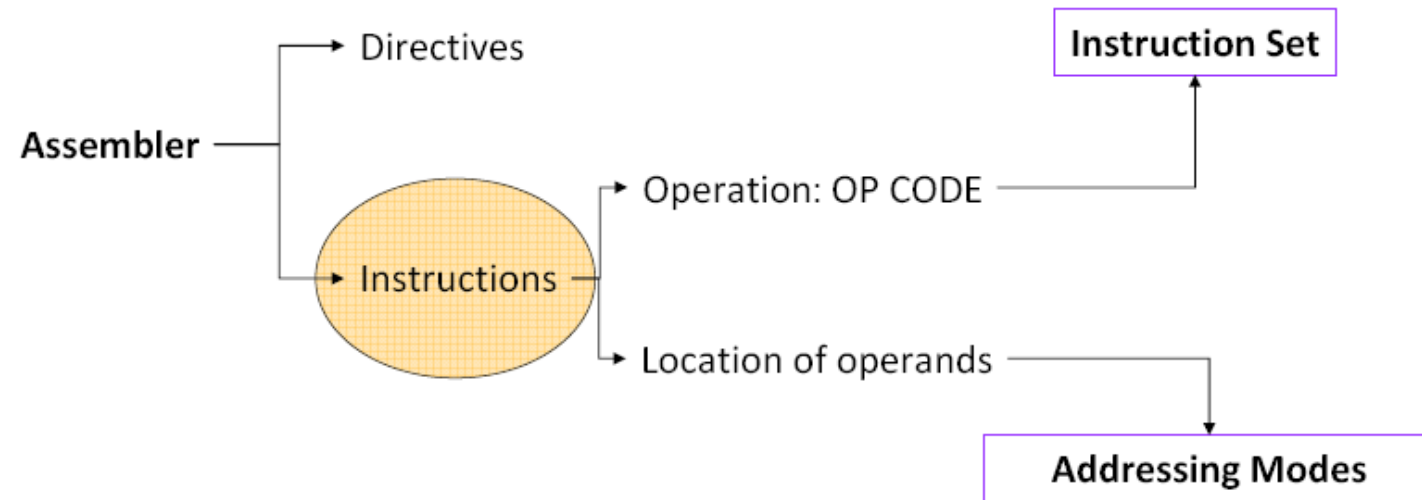
TSTFSZ f,a: test f, skip if 0

2.6 Pipelining and execution time

Changes in program counter:

- Microcontroller executes instruction sequentially in normal condition.
- PIC18 has a 21-bit program counter (PC) which is divided into three registers: PCL, PCH, and PCU.
- PCL can be accessed directly. However, PCH and PCU are not directly accessible.
- One can access the values of PCH and PCU indirectly by accessing the PCLATH and PCLATU.
- Reading the PCL will cause the values of PCH and PCU to be copied into the PCLATH and PCLATU.
- Writing the PCL will cause the values of PCLATH and PCLATU to be written into the PCH and PCU.
- In normal program execution, the PC value is incremented by either 2 or 4.
- To implement a program loop, the processor needs to change the PC value by a value other than 2 or 4.

2.7 Assembly programming tips



There are six basic types of directives provided by the assembler.

- Control Directives
- Conditional Assembly Directives
- Data Directives
- Listing Directives
- Macro Directives
- Object File Directives

2.7 Assembly programming tips

Examples of use of the directives:

`#define <name> [<string>]`

`#define PORTA 80`

This directive defines a text substitution string. Whenever <name> is encountered in the assembly code, <string> will be substituted.

`#include <include_file>`

`#include <p18f2525.inc>`

This directive includes additional source file. The specified file is read in as source code. The effect is the same as if the entire text of the included file were inserted into the file at the location of the include statement.

`[<label>] org <expr>`

`Reset ORG 0000h`

This directive sets the program origin for subsequent code at the address defined in <expr>.

2.7 Assembly programming tips

Case issue:

The PIC18 instructions can be written in either uppercase or lowercase.

MPASM allows the user to include “p18Fxxxx.inc” file to provide register definitions for the specific processor.

All special function registers and bits are defined in uppercase.

The **convention** followed in this text is: using **lowercase** for instructions and directives, using **uppercase** for special function registers.

Program begin and end:

START

We fix the location of instructions in memory by the directive ORG

```
org 0x0000h  
bra main
```

STOP

Program can NOT wander around any memory location.
Execution must be limited to program lines written by user.

```
loop  
    bra loop  
end
```

2.7 Assembly programming tips

Assembly programming template:

```
                org    0x0000    ; program starting address after power on reset
                goto   start

                org    0x08
                ...          ; isr code here
                retfie         ; high-priority interrupt service routine

                org    0x18
                ...          ; isr code here
                retfie         ; low-priority interrupt service routine

start          ...          ; your program here
                ...
                end
```

Semicolon (“;”) is used to insert comments. The compiler requires “end” at the end of the file.

2.7 Assembly programming tips

Some code structures in assembler:

IF/ELSE structure

```

        btfsc STATUS,Z    ; flag Z test
        goto Zset

Zclear  . . .             ; code for Z=0
        goto Zdone

Zset    . . .             ; code for Z=1
Zdone   . . .             ; We're done
  
```

LOOP structure:

```

i_cnt   equ    PRODL    ; use PRODL as loop count
N       equ    20       ; loop limit

i_loop  clrf     i_cnt, A
        ...
        incf     i_cnt, A ; i_cnt is incremented in the loop
        movlw    N
        cpfseq   i_cnt,A ; compare i_cnt with WREG and
                          ; skip if equal
        goto     i_loop  ; executed when i_cnt ≠ loop limit
  
```

2.7 Assembly programming tips

Sample program 1

Write a program that adds the three numbers stored in data registers at 0x20, 0x30, and 0x40 and places the sum in data register at 0x50.

```
#include <p18F45K22.inc>          ; can be other processor
    org      0x00
    goto     start
    org      0x08
    retfie
    org      0x18
    retfie
start    movf   0x20,W,A   ; WREG ← [0x20]
        addwf  0x30,W,A   ; WREG ← [0x20] + [0x30]
        addwf  0x40,W,A   ; WREG ← [0x20] + [0x30] + [0x40]
        movwf  0x50,A      ; 0x50 ← sum (in WREG)
        end
```

2.7 Assembly programming tips

Sample program 2

Write a program to add two 24-bit numbers stored at 0x10~0x12 and 0x13~0x15 and leave the sum at 0x20~0x22.

```
#include <p18F45K22.inc>          ; can be other processor

        org      0x00
        goto     start
        org      0x08
        retfie
        org      0x18
        retfie
start    movf      0x10,W,A  ; WREG ← [0x10]
        addwfc    0x13,W,A  ; WREG ← [0x13] + [0x10]
        movwf     0x20,A      ; 0x20 ← [0x10] + [0x13]
        movf      0x11,W,A  ; WREG ← [0x11]
        addwfc    0x14,W,A  ; WREG ← [0x11] + [0x14] + C flag
        movwf     0x21,A      ; 0x21 ← [WREG]
        movf      0x12,W,A  ; WREG ← [0x12]
        addwfc    0x15,W,A  ; WREG ← [0x12] + [0x15] + C flag
        movwf     0x22,A      ; 0x22 ← [WREG]
        end
```

2.7 Assembly programming tips

Sample program 3

Write a program to compute $1 + 2 + 3 + \dots + n$ and save the sum at 0x00 and 0x01.

```
#include <p18F45K22.inc>          ; can be other processor

n equ    D'50'
sum_hi set    0x01      ; high byte of sum
sum_lo set    0x00      ; low byte of sum
i set     0x02          ; loop index i
org      0x00          ; reset vector
goto     start
org      0x08
retfie
org      0x18
retfie

start    clrf      sum_hi,A    ; initialize sum to 0
         clrf      sum_lo,A    ;
         clrf      i,A         ; initialize i to 0
         incf      i,F,A       ; i starts from 1

sum_lp   movlw     n           ; place n in WREG
         cpfsgt    i,A         ; compare i with n and skip if i > n
         bra       add_lp      ; perform addition when i ≤ 50
         bra       exit_sum     ; it is done when i > 50

add_lp   movf      i,W,A       ; place i in WREG
         addwf     sum_lo,F,A  ; add i to sum_lo
         movlw     0
         addwfc    sum_hi,F,A  ; add carry to sum_hi
         incf      i,F,A       ; increment loop index i by 1
         bra       sum_lp

exit_sum nop
         bra       exit_sum
         end
```

2.7 Assembly programming tips

Sample program 4

Make 1 ms delay for a 40MHz PIC.

[illegible]